4-4 · Trino 기반 S3 SQL 엔진 구축 · 이론 1H

분산 SQL 엔진의 이해

저장과 계산이 갈라지면 데이터를 옮기지 않는다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① Trino 의 코디네이터·워커 구조와 커넥터 개념을 말할 수 있다

② 저장과 계산이 분리된 구조의 장점을 설명한다

③ 커넥터와 카탈로그를 구별하고, 파일 이름이 카탈로그 이름임을 안다

④ Spark 의 demo 와 Trino 의 iceberg 가 같은 실체임을 설명한다

다음 1.5H 실습에서 Spark 가 쓴 테이블을 Trino 가 그대로 읽는지 확인합니다

한 몸과 분리 — 다섯 가지 차이

	한 몸 (MySQL 등)	분리 (레이크하우스)
데이터 위치	DB 서버의 디스크	오브젝트 스토리지
엔진을 늘리려면	서버를 키운다	엔진만 더 띄운다
다른 도구가 쓰려면	복사해 간다	같은 것을 그대로 읽는다
엔진을 바꾸려면	데이터 이전	설정만 바꾼다
비용	저장·계산이 묶여 있다	따로 조절한다

MySQL 은 자기 파일을 자기가 읽는다 — 커지면 서버를 통째로 키우고, 다른 도구는 복사해 갔다

쓸 때만 계산 자원을 켠다 — 새벽엔 Spark 를 크게, 낮엔 Trino 만 작게. 데이터는 그대로

데이터를 갖지 않고 매번 원본에서 읽는 엔진

1. **Presto 의 계보**
   Presto 창시자들이 독립해 이어가는 프로젝트(구 PrestoSQL) — 강의계획서의 Trino(Presto) 표기

2. **데이터를 저장하지 않는다**
   SQL 이 올 때마다 원본에서 읽어 계산하고 결과만 돌려준다

3. **대화형 질의에 맞춰져 있다**
   사람이 앉아서 결과를 기다리는 몇 초~몇 분짜리 쿼리

4. **여러 소스를 한 SQL 로**
   Iceberg 테이블과 MySQL 테이블을 함께 조인할 수 있다

Spark 와 Trino — 만들 때와 분석할 때

	Spark	Trino
주 용도	무거운 배치 변환·쓰기	대화형 조회·집계
실패 시	단계별로 재시도	쿼리 전체 재실행
쓰기	강함 (MERGE·UPDATE)	가능하지만 제한적
지연	수 분~수 시간	수 초~수 분
프로그래밍	Python·Scala API	SQL 중심

파이프라인이 데이터를 만들 때는 Spark, 사람이 분석할 때는 Trino — 4-3 과 4-4 가 그 두 자리

SQL 하나가 처리되는 길

1. **코디네이터가 SQL 을 받는다**
   실행 계획을 세우고, 워커에게 일을 나눠 주고, 결과를 모은다

2. **워커가 읽고 계산한다**
   실제로 스토리지에서 읽고 계산 — 여러 대를 두면 그만큼 나눠 처리

3. **실습은 단일 컨테이너**
   코디네이터가 워커를 겸임 — 5주차 Spark 의 드라이버·익스큐터와 같은 구조

분산 처리의 공통 모양

코디네이터

+ 워커

설정 파일에서 헷갈리는 두 낱말

커넥터

이 종류의 데이터 소스와 대화하는 방법

- iceberg · postgresql · mysql · kafka 등
- 데이터 소스 종류마다 커넥터 하나
- 설정 파일의 connector.name 이 이것

VS

카탈로그

그 커넥터로 연결한 데이터 소스 하나의 설정

- etc/trino/catalog/ 의 properties 파일 하나
- 파일 이름 = 카탈로그 이름 → iceberg.스키마.테이블
- 같은 커넥터로 카탈로그 여러 개도 가능(운영용·개발용)

iceberg.properties — 「어디 있나」와 「직접 읽기」

1. **connector.name=iceberg**
   어떤 커넥터를 쓸지 — 이 값이 없으면 카탈로그가 로드되지 않는다

2. **iceberg.catalog.type=rest**
   Iceberg 카탈로그 종류 — Spark 쪽과 같아야 한다

3. **iceberg.rest-catalog.uri**
   (1) 「어디 있나」를 묻는 곳 — 카탈로그 서비스 주소

4. **s3.endpoint**
   (2) 파일을 직접 읽는 곳 — S3 주소. 실습 1단계에서 이 파일을 씁니다

주석은 따로 한 줄

줄 끝 # 은

값으로 읽힌다

5. 이름 구조

iceberg.lakehouse.orders — 세 칸의 뜻

Trino	뜻	이번 실습에서
카탈로그	연결한 데이터 소스	iceberg
스키마	그 안의 데이터베이스	lakehouse
테이블	표	orders
전체 이름	카탈로그.스키마.테이블	iceberg.lakehouse.orders

Spark 의 demo.lakehouse.orders 와 앞 이름만 다르다 — 각 엔진이 카탈로그에 붙인 별명, 실체는 하나

정리

이 1시간에서 꼭 가져갈 것

1. 저장과 계산이 분리되면 엔진을 늘리거나 바꿔도 데이터를 안 옮긴다
2. Trino 는 데이터를 저장하지 않고 매번 원본에서 읽어 계산한다
3. 무거운 쓰기는 Spark, 대화형 조회는 Trino
4. 커넥터는 대화 방법 · 카탈로그는 연결한 소스 하나 · 파일 이름 = 카탈로그 이름
5. Spark 의 demo 와 Trino 의 iceberg 는 같은 카탈로그의 다른 별명이다

4-4 · 이론 1H 마무리

다음은 Trino 컨테이너 연결 실습 (1.5H)

카탈로그 설정 파일을 만들고 Trino 를 띄워, Spark 가 쓴 테이블이 그대로 보이는지 확인합니다

- ✓ **catalog/iceberg.properties 열 줄 작성**
- ✓ **compose 교체 · up -d 뒤 「준비 완료」 대기**
- ✓ **SHOW TABLES 로 orders 확인**
- ✓ **두 엔진의 count·sum 이 같은지 대조**

준비할 것

4-3 의 컨테이너 4개 · 8080 포트 비우기